

BE : UNE INTRODUCTION À L'ANALYSE EN COMPOSANTES INDÉPENDANTES (ICA).

Consignes Générales :

- Par groupe de 5 à 6 étudiants maximum, vous devez rédiger un rapport dans lequel vous répondrez aux questions du sujet et plus globalement au problème demandé. La qualité de rédaction ainsi que tous les compléments que vous apporterez (réflexions, analyses des réponses, tests sur d'autres données, etc...) seront fortement pris en compte. Ce rapport devra être rédigé sous un traitement de texte (les pages manuscrites ne sont pas acceptées) et rendu sous la forme d'un fichier .pdf.
- Toutes les fonctions programmées le seront en Python et seront abondamment commentées. Le rendu du projet se fera sous la forme d'un fichier .zip contenant le rapport ainsi que l'ensemble des programmes et données nécessaires à l'exécution des programmes.
- Les fonctions programmées doivent l'être dans un fichier librairie ayant pour nom : **Ma315_BE_lib.py**. Dans ce fichier les différentes fonctions doivent être regroupées dans des blocs (commande : `#%%`) correspondants aux parties du sujet.
- Les applications numériques des fonctions du sujet seront dans un fichier ayant pour nom : **Ma315_BE_main.py**. Dans ce fichier les différents cas d'étude doivent être regroupés dans des blocs séparés.
- Les différentes données utilisées devront être enregistrées dans un dossier ayant pour nom **Data**. Ce dossier doit être mis dans le même répertoire que vos fichiers **Ma315_BE_lib.py** et **Ma315_BE_main.py**.
- Les noms des différents membres du groupe doivent apparaître en commentaire en début de fichier.
- Le non-respect des consignes précédentes entraînera des points de pénalité pouvant être conséquents...
- Concernant Python, le rendu attendu est l'utilisation des fonctions de bases des librairies Numpy, Matplotlib et OpenCV. Néanmoins l'utilisation de fonctions plus avancées qui améliorent les rendus graphiques par exemple et/ou la programmation en terme de classe seront bien évidemment acceptées et seront valorisées.
- Les questions d'analyse de résultats sont généralement ouvertes. La qualité de vos réflexions sera fortement prise en compte pour la notation de ces questions.
- **Il est impératif de lire le sujet en entier.**
- **Enfin on rappelle qu'un BE est un examen et comme tout examen il arrive de ne pas pouvoir répondre à toutes les questions.**

Dans ce bureau d'étude nous allons étudier le difficile problème de la séparation « aveugle » des sources que nous allons expliquer dans la suite. Lorsque l'on observe un jeu de données (par exemple issues de capteurs comme des antennes, microphones ou des images, etc...), ces données observées peuvent être issu d'un mélange de signaux sources. Le problème est alors d'arriver à isoler les signaux sources à partir des données observées mélangées. L'exemple paradigmatique est celui du « *party cocktail problem* », où l'on place plusieurs microphones dans une salle, ces microphones enregistrent des mélanges de conversation et le but de la séparation « aveugle » des sources est d'isoler les différentes voix (= les sources). Ce type de problème, dans sa formulation la plus générale, n'a en général aucune solution théorique mais il est néanmoins possible en ajoutant des hypothèses de trouver des « solutions acceptables ». Nous allons voir une catégorie d'approche qui en imposant certaines conditions permet d'avoir des résultats (très) corrects : l'**analyse en composantes indépendantes** (*independent component analysis* que nous abrègerons par **ICA**). Dans cette approche les signaux sources inconnues sont modélisés par un vecteur aléatoire : $S := (S_1, \dots, S_m)^T$, les signaux captés par un vecteur aléatoire : $X := (X_1, \dots, X_m)^T$. On supposera que le mélange est linéaire i.e. qu'il existe une matrice carrée inversible $A \in \mathcal{M}_{m,m}(\mathbb{R})$ telle que :

$$X = AS$$

On appellera X le vecteur des mélanges de S . Nous avons fait ici l'hypothèse simplificatrice qu'il y a autant de sources que de mélanges i.e. que A est carrée et que le mélange n'est pas « destructeur » en en supposant que A est inversible. Le but de l'ICA est déterminer la matrice A (ou son inverse) et S . Ainsi A et S sont inconnus. On voit que même l'hypothèse d'un mélange linéaire conduit à un problème où le nombre d'inconnues est bien plus grand de le nombre de données disponibles...

Dans les figures suivantes on donne un exemple du problème de l'ICA où les signaux sont des images (l'exemple des signaux sonores est « moins visuel »...) et où les images sont vectorialisées i.e. en les mettant sous forme de vecteur ligne. On considère ici le cas où $m = 2$.

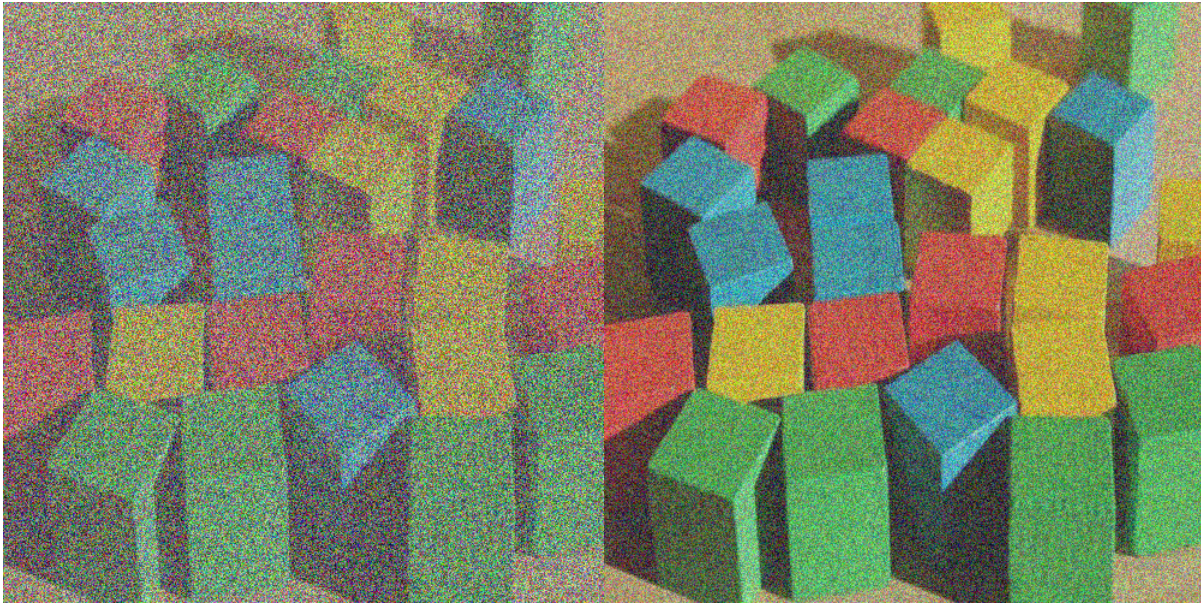


FIGURE 1 – On capte deux images différentes d'une même scène X_1 et X_2 . Ces deux images sont bruitées et l'on suppose qu'elles sont les mélanges (linéaires) d'une image correcte S_1 et d'un bruit S_2 . Le but de l'ICA est de retrouver ce S_1 (et S_2).

On va faire l'hypothèse suivante : les sources S_1 et S_2 sont des variables aléatoires indépendantes. Nous verrons qu'en utilisant un critère de non-gaussianité i.e. la recherche de v.a.r. les « moins gaussiennes possibles » nous allons pouvoir « supprimer » le bruit et retrouver l'image S_1 comme le montre la figure suivante qui est le résultat de l'application d'un des algorithmes que nous allons programmer dans ce BE.

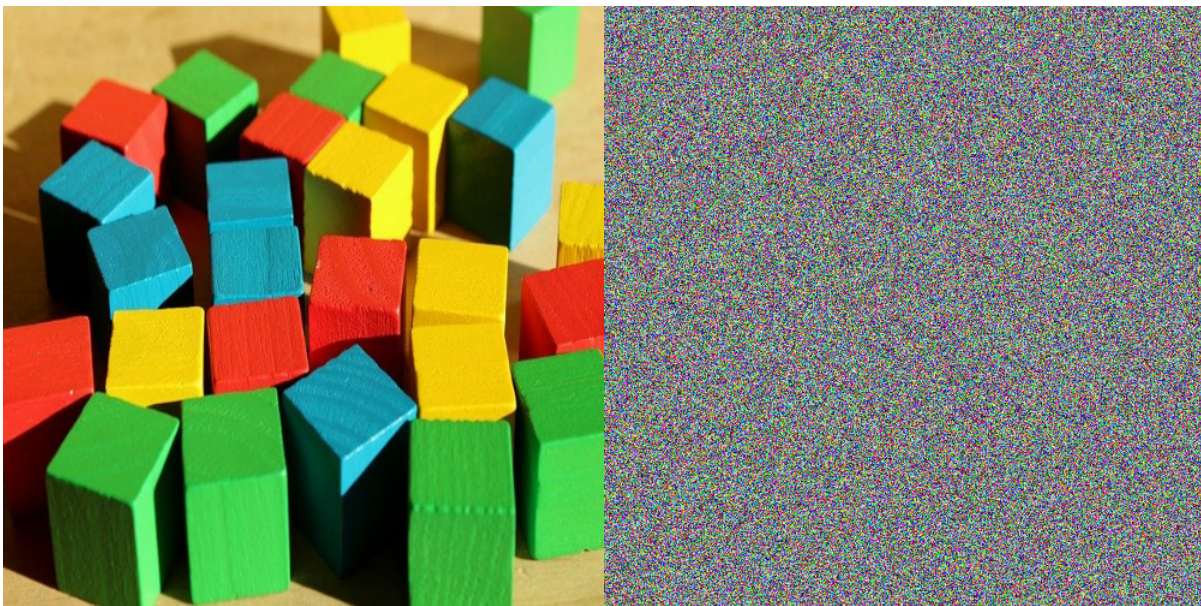


FIGURE 2 – Résultat de l'algorithme du BE. On retrouve l'image nette et le bruit séparés. Impressionnant non ? !

Ce bureau d'études se découpe en 3 parties, chacune de ces parties étudient un aspect de la résolution de l'ICA. La partie 1 et 2 sont globalement indépendantes et la partie 3 est une synthèse de ces deux parties. Attention seulement : il faut noter que l'algorithme que nous allons voir est le plus « simple » parmi les méthodes pour résoudre l'ICA et par suite souvent le moins efficace pour cette résolution, il ne faudra donc pas s'attendre à des miracles quant à l'efficacité de notre algorithme.

1 Préparation de l'ICA et blanchiment des données

On rappelle les hypothèses de l'introduction. On se donne $S := (S_1, \dots, S_m)^T$ et $X := (X_1, \dots, X_m)^T$ deux vecteurs aléatoires (considérés sous la forme de vecteurs colonnes), une matrice carrée inversible $A \in \mathcal{M}_{m,m}(\mathbb{R})$ telle que :

$$X = AS$$

Pour M une matrice dont les coefficients sont des v.a.r. on notera $E(M)$ la matrice réelle dont les coefficients sont les espérances des v.a.r i.e. pour tout i, j :

$$E(M)_{ij} := E(X_{ij})$$

Le but du blanchiment (*whitening*) des données est de trouver une transformation linéaire B telle que le vecteur aléatoire $Z := BX$ possède une matrice de covariance égale à la matrice identité.

Dans toute la suite on supposera que le vecteur X est centré i.e. que $E(X) = 0_m$.

1. Pourquoi peut-on dire qu'un vecteur aléatoire ayant une matrice de covariance diagonale a ses composantes décorrélées ?
2. Montrer que l'hypothèse $E(X) = 0_m$ implique $E(S) = 0_m$.
3. Justifier que la matrice de covariance $Cov(X)$ est donnée par :

$$Cov(X) = E(XX^T)$$

4. En utilisant les propriétés de la matrice de covariance justifier qu'il existe une matrice U orthogonale et D une matrice diagonale à coefficient positif ou nul telle que :

$$E(XX^T) = UDU^T$$

5. En déduire que le vecteur aléatoire : $Z := D^{-\frac{1}{2}}U^T X$ est un vecteur blanchi i.e. que :

$$E(ZZ^T) = I_m$$

où $D^{-\frac{1}{2}}$ est la matrice diagonale dont les coefficients sont soit nuls si les D_{ii} sont nuls, soit égaux à $D_{ii}^{-\frac{1}{2}}$ dans le cas où les D_{ii} sont non nuls.

6. On s'intéresse alors au problème suivant :

$$Z = \tilde{A}S$$

où $Z = D^{-\frac{1}{2}}U^T X$ et $\tilde{A} := D^{-\frac{1}{2}}U^T A$.

- (a) Démontrer que :

$$I_m = \tilde{A}Cov(S)\tilde{A}^T$$

- (b) En déduire que si S est :

- un vecteur aléatoire aux composantes mutuellement indépendantes,
- tel que pour tout $i \in \llbracket 1, m \rrbracket$: $Var(S_i) = 1$

Alors la matrice $\tilde{A}$ est une matrice orthogonale.

7. **Applications.** Dans la pratique ce ne sont pas les variables aléatoires que nous allons avoir directement mais des réalisations de ces variables aléatoires. On suppose pour chaque X_i de X que l'on possède une réalisation d'un n -échantillon, $n \in \mathbb{N}^*$ i.e. que l'on a un vecteur ligne :

$$x_{i\bullet} := (x_{i1}, \dots, x_{in})$$

On notera $X_{(r)}$ la matrice de taille (m, n) dont la ligne i est donnée par $x_{i\bullet}$. On appellera une telle matrice $X_{(r)}$, **une matrice de réalisation** de X .

- (a) Montrer que :

$$\widehat{Cov(X)} := \frac{1}{n-1} X_{(r)} X_{(r)}^T$$

est une approximation de la matrice de covariance de X .

- (b) En utilisant les questions précédentes, montrer qu'il existe une matrice diagonale D et une matrice orthogonale U telle que :

$$Z_{(r)} := D^{-\frac{1}{2}}U^T X_{(r)}$$

soit la matrice de réalisation d'un vecteur aléatoire blanc i.e. que :

$$\widehat{Cov(Z)} \simeq I_m$$

(c) Créer deux fonctions pythons :

- i. Pour la première : **center**(Xr), qui prend en entrée une matrice de réalisation de taille (m, n) et qui renvoie une version centrée de celle-ci. Par abus de notation on notera encore Xr (ou $X_{(r)}$) le résultat du centrage.
- ii. Enfin une fonction : **whitening**(Xr) qui prend en entrée une matrice de réalisation centrée de taille (m, n) et qui renvoie la matrice de réalisation blanchie $Z_{(r)}$ de la question précédente.

(d) Tester votre fonction de blanchiment (whitening) en utilisant le fichier **blanchiment_exemple1.npy** (que l'on chargera avec np.load).

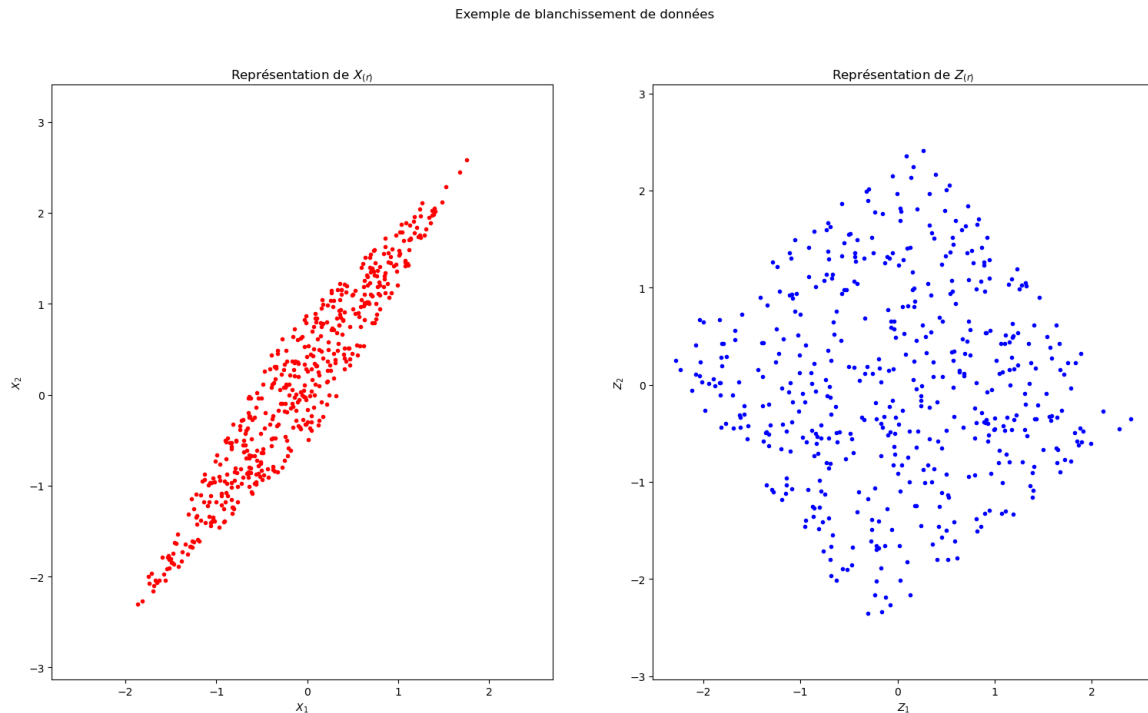


FIGURE 3 – Un exemple de blanchiment de données.

(a) Considérons une image couleur *img*. Avec OpenCV cette image est une « hyper »-matrice de dimension $(m, n, 3)$, chaque couche étant associée aux intensités des couleurs trois couleurs rouge, vert et bleu.

- i. Créer une fonction **vect_image**(img) qui prend en entrée une image couleur et renvoie en sortie une matrice de dimension $(3, m \times n)$ où chaque ligne de cette matrice correspond à la version vectorisée (en ligne) de chaque couche des couleurs rouge, vert et bleu respectivement.
- ii. Appliquer l'opération de blanchiment à la sortie de la fonction précédente. **Attention on rappelle qu'il faut penser à convertir en float les images avant de faire des calculs et de re-converter les résultats des calculs en uint8** (et remettre à la taille) avant d'afficher les images. On pourra également utiliser la commande `cv2.normalize(votre_sortie, None, 0,255, cv2.NORM_MINMAX)` afin de remettre à l'échelle les données.

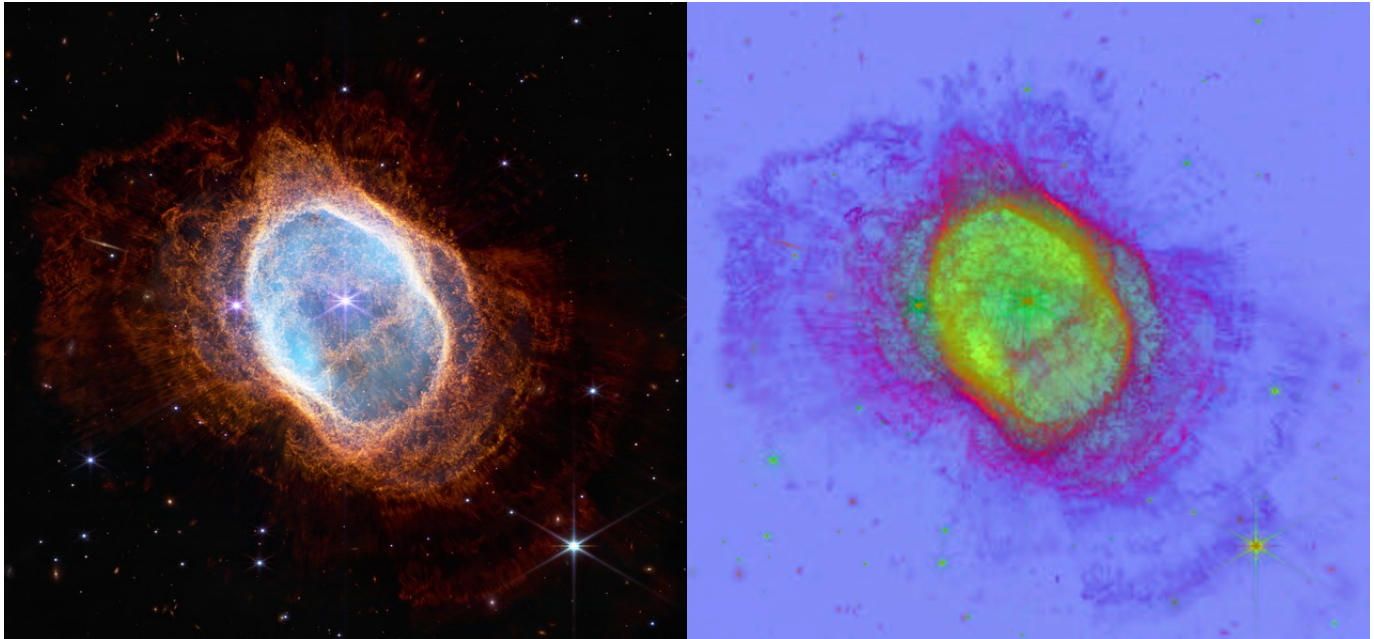


FIGURE 4 – à gauche l'image originale de la nébuleuse NGC 3132, à droite sa version « blanchie ».

Pouvez-vous expliquer ce que vous observez ?

- (b) En réutilisant les codes donnés en Ma312, considérons à présent un son wav à deux canaux où les deux canaux ne sont pas identiques. Effectuer une opération de blanchissement sur les sons wav de votre choix. Quel(s) effet(s) cela a-t-il sur le son ? Écouter séparément les deux canaux ainsi modifiés. Comme pour les images il faut penser à convertir vos données en float avant tout traitement puis à les re-converter au bon format une fois les calculs effectués. De plus il faudra transposer les sons importés et avant écoute du son blanchie il est conseillé d'utiliser la fonction de OpenCV :

```
zrenorm=cv2.normalize(z,None,np.min(x),np.max(x),cv2.NORM_MINMAX)
```

où z est la matrice (à deux ligne) blanchie et x est le son importé. On pourra également tracer le profil du son original et le comparer avec sa version blanchie. Vous pouvez tester votre code sur le son son1.wav ou son2.wav (par exemple) mais surtout sur des sons de votre choix.

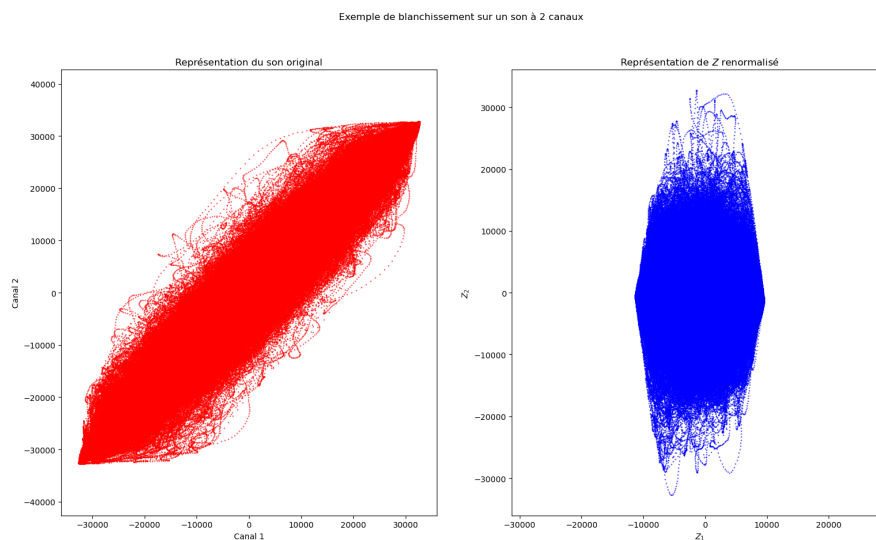


FIGURE 5 – à gauche son original, à droite sa version « blanchie ».

2 Un critère de non-gaussianité : le kurtosis

À cause du théorème central limite, la loi normale joue un rôle prépondérant en statistique : c'est généralement la loi étalon dans beaucoup de cas pratique. L'heuristique que nous allons utiliser afin de résoudre le problème de la séparation « aveugle » de sources est de faire l'hypothèse que dans la pratique les variables aléatoires incarnant les sources $S := (S_1, \dots, S_m)^T$ doivent être moins « gaussiennes » que les versions mélangées $X := (X_1, \dots, X_m)^T$ où l'on rappelle que X et S sont en relation par :

$$X = AS$$

où A est une matrice carrée inversible. Dans cette partie nous allons commencer par vérifier expérimentalement cette heuristique à l'aide de deux critères : le **coefficient d'asymétrie** (skewness) et le **coefficient d'aplatissement** plus communément appelé le **kurtosis**.

Soit Y une variable aléatoire ayant des moments d'ordre 3, on définit le **coefficient d'asymétrie** par :

$$As(Y) := E \left(\left(\frac{Y - \mu}{\sigma} \right)^3 \right)$$

où $\mu = E(Y)$ et $\sigma = \sqrt{Var(Y)}$ et on choisira comme estimateur (naturel) pour un n -échantillon $(Y_1, \dots, Y_n)$ de loi parente Y :

$$\widehat{As}(Y) := \frac{\frac{1}{n} \sum_{i=1}^n (Y_i - \bar{Y}_n)^3}{\left(\frac{1}{n-1} \sum_{i=1}^n (Y_i - \bar{Y}_n)^2 \right)^{\frac{3}{2}}}$$

De même on définira le **kurtosis** (normalisé) par :

$$kurt(Y) := E \left(\left(\frac{Y - \mu}{\sigma} \right)^4 \right) - 3$$

et comme estimateur (naturel) :

$$\widehat{kurt}(Y) := \frac{\frac{1}{n} \sum_{i=1}^n (Y_i - \bar{Y}_n)^4}{\left(\frac{1}{n-1} \sum_{i=1}^n (Y_i - \bar{Y}_n)^2 \right)^2} - 3$$

- Dans chacun des cas suivants calculer pour Y une v.a.r sont coefficient d'asymétrie et son kurtosis dans le cas où :
 - Y suit une loi normale,
 - Y suit une loi uniforme,
 - Y suit une loi exponentielle,
 - Y suit une loi de Pareto avec $\alpha > 4$.
- En utilisant les valeurs calculées précédemment et en faisant varier les paramètres des lois précédentes, justifier empiriquement (ou pourra utiliser les graphes des densités) les noms de coefficient d'asymétrie et coefficient d'aplatissement (pour le kurtosis).
- En utilisant des simulations de taille n pour différentes valeurs de $n \in \mathbb{N}^*$ de chacune des lois de la question 1, comparer la valeur des calculs théoriques et les valeurs données par les estimateurs. Pour cela vous définirez deux fonctions sous python : `esti_asy(x)` et `esti_kurt(x)`, chacune prenant en entrée un array de taille $(n,)$ et renvoyant en sortie les valeurs données par les formules des estimateurs précédents. Que pouvez-vous en déduire ?
- À présent, nous allons tester notre heuristique sur des données réelles. Une manière simple de trouver des variables aléatoires assez générales est de considérer des images (on pourrait également prendre des sons en exemple ou d'autres types de données, à vous de tester). Pour tester cela vous devez :
 - Charger deux images en noir et blanc avec OpenCV (option ,0 de `cv2.imread`). Penser à les convertir en float.
 - À l'aide d'un `cv2.resize` redimensionner les images pour qu'elles soient de même taille (par exemple 500,500),
 - Vectoriser en array de type $(n,)$ ces deux images, notons les $s1$ et $s2$. Calculer grâce à la question précédente leurs coefficients d'asymétrie et leurs kurtosis.

- En utilisant les commandes numpy `np.mean` et `np.std`, centrer-réduire $s1$ et $s2$. Tracer sur un même graphe l'histogramme de chacune de ces variables avec l'option `bins=256` et `density=True` ainsi que la densité de la loi normale centrée-réduite.
- Définir un mélange de $s1$ et $s2$: $s_m = \alpha_1 s1 + \alpha_2 s2$ avec α_1 et α_2 des réels. Calculer le coefficient d'asymétrie et le kurtosis de s_m . Tracer également l'histogramme de sa version centrée-réduite.

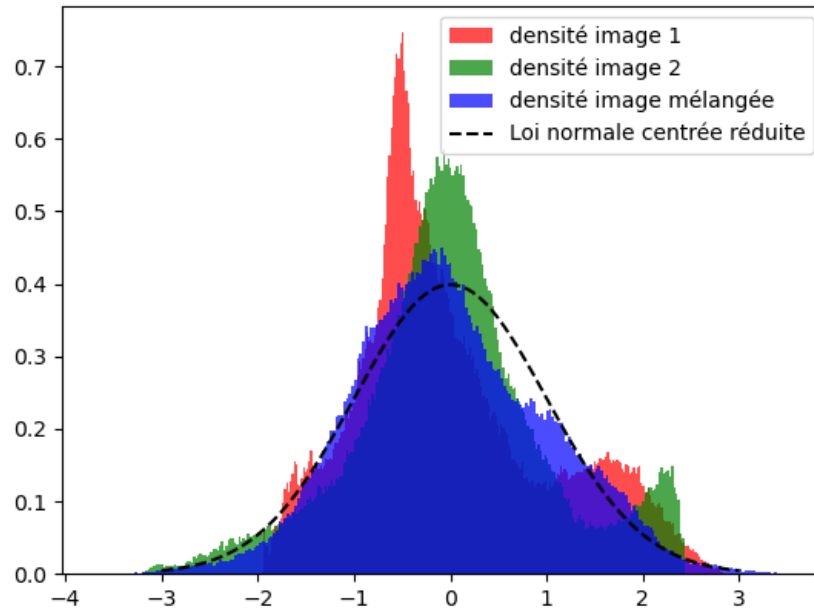


FIGURE 6 – Exemple de sortie avec les images *fraise.jpg* et *chien.jpg* et pour un mélange avec $\alpha_1 = \alpha_2 = 1$.

En essayant différent mélange, que remarquez-vous sur la valeur du kurtosis ?

- En utilisant les commandes données en Ma312, à l'image de la question précédente tester l'heuristique de cette partie sur des mélanges de sons.
- Un autre test simple pour « tester » la normalité d'une distribution de valeurs est celui de la droite d'Henry. Supposons que l'on connaisse la moyenne μ et la variance σ^2 d'une loi de probabilité X alors si X suit une loi normale $\mathcal{N}(\mu, \sigma^2)$ pour tout x réel on a :

$$P(X \leq x) = P\left(\frac{X - \mu}{\sigma} \leq \frac{x - \mu}{\sigma}\right)$$

- Quel loi suit la variable aléatoire : $Y = \frac{X - \mu}{\sigma}$?
- Notons Φ la fonction de répartition de Y et $t_i = \frac{x_i - \mu}{\sigma}$ pour x_i des réalisation de X . Alors la courbe formée des t_i en fonction des x_i est une droite. Ainsi le test de la droite d'Henry consiste pour une réalisation de taille n d'une v.a.r : $x = (x_1, \dots, x_n)$ à la procédure suivante. Créer une fonction `test_henry(x)` qui réalise les étapes qui suivent :
 - On commence par donner une estimation de μ et de σ . On pourra prendre comme estimateur de σ :

$$\hat{\sigma}_n := \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X}_n)^2}$$

Bonus : Montrer que cet estimateur est en réalité biaisé. On peut montrer (mais cela est difficile..) que l'estimateur de l'écart-type suivant est sans biais pour une loi normale centrée réduite :

$$\widetilde{\sigma}_n := \frac{\hat{\sigma}_n}{\sqrt{\frac{2}{n-1} \frac{\Gamma(\frac{n}{2})}{\Gamma(\frac{n-1}{2})}}}$$

où Γ note la fonction Gamma d'Euler.

- ii. Puis on calcule une estimation de : $P(X \leq x_i)$ et on en déduit t_i tel que :

$$\Phi(t_i) = P(X \leq x_i)$$

La fonction réciproque de la loi normale centrée réduite s'appelle la **fonction probit** et s'écrit pour $p \in]0, 1[$:

$$\text{probit}(p) := \sqrt{2} \text{erf}^{-1}(2p - 1)$$

où erf^{-1} est la fonction inverse de la fonction erreur. On pourra utiliser pour trouver t_i l'implémentation dans la librairie Scipy de la fonction inverse de la fonction erreur : `scipy.special.erfinv`.

- iii. On trace alors la courbe engendrée par les couples (x_i, t_i) et la courbe équivalente pour une loi normale centrée réduite.
- iv. On calcule enfin l'écart entre les couples (x_i, t_i) et les couples théoriques $(x_i, \tilde{t}_i)$ (on pourra utiliser la norme 2 de la différence et normaliser par la taille de l'échantillon) et afficher le résultat sur le titre du graphe précédent.

Plus les points s'alignent sur la droite donnée par la loi normale centrée réduite, plus l'on peut espérer que notre réalisation vient d'une loi normale. Au contraire plus la courbe engendrée par les couples (x_i, t_i) s'éloigne de cette droite plus l'on peut parier sur la non-normalité .

7. Utiliser le test de la droite d'Henry pour tester l'heuristique de cette partie i.e. que sur des réalisations d'échantillons le fait que des combinaisons linéaires de variables aléatoires (=mélanges) sont plus gaussiennes, i.e. suivent « mieux » la loi normale, que les sources dont sont issus ces mélanges. On testera comme précédemment sur des images ainsi que sur des mélanges de fichiers sonores (de taille raisonnable....).

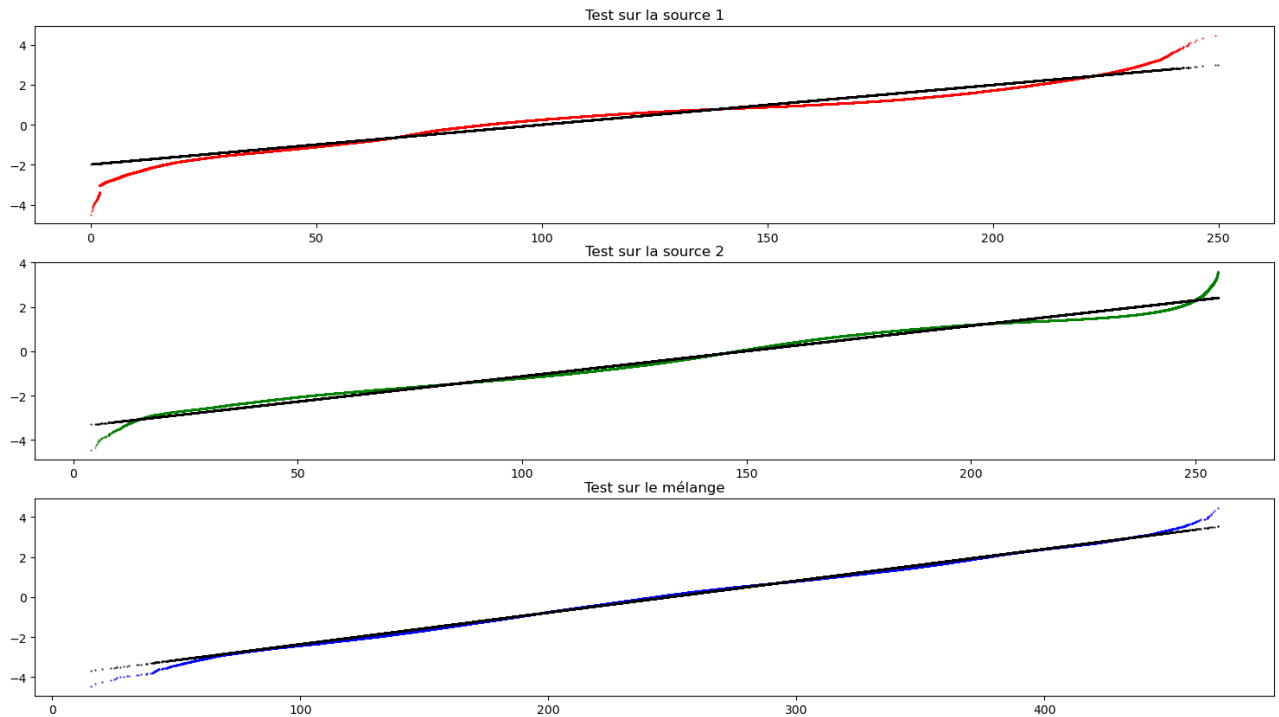


FIGURE 7 – Exemple de sortie avec les images *fraise.jpg* et *chien.jpg* en noir et blanc et pour un mélange de ces deux images avec $\alpha_1 = \alpha_2 = 1$. Les deux premiers graphes sont les tests d'Henry sur les deux sources. Le graphe du bas est le test sur le mélange. On voit que ce dernier s'écarte moins de la droite référence et par suite semble être « plus » gaussienne.

3 Résolution de l'ICA avec maximisation du kurtosis

La partie précédente nous guide (de manière empirique) vers un choix de critère qui peut potentiellement nous aider résoudre le problème de l'ICA. On rappelle que l'on recherche à résoudre :

$$X = AS$$

où seul X est connu. Grâce à la première partie nous avons montré que l'on peut toujours se ramener, après blanchiment, à la résolution d'un problème de la forme :

$$Z = \tilde{A}S$$

où Z est connue et de matrice de covariance égale à l'identité et où l'on a fait l'hypothèse que les composantes du vecteur aléatoire S sont mutuellement indépendante et de variance égale à 1. De plus nous avons vu que $\tilde{A}$ est une matrice orthogonale. En notant : $W = \tilde{A}^T$, l'inverse de $\tilde{A}$, il nous faut donc trouver pour tout $i \in \llbracket 1, m \rrbracket$:

$$S_i = W_{i\bullet}Z$$

où $W_{i\bullet}$ note le vecteur ligne de la i -ème ligne de la matrice W . La partie 2 nous guide alors vers la stratégie suivante : nous allons chercher pour tout $i \in \llbracket 1, m \rrbracket$ à maximiser la valeur absolue du kurtosis :

$$|kurt(S_i)| = |kurt(W_{i\bullet}Z)|$$

où le vecteur ligne $W_{i\bullet}$ est inconnu sous la contrainte que la matrice W soit orthogonale. Le but est de trouver les S_i les moins gaussiens possibles.

Attention cependant les solutions que nous allons trouver ne sont pas « uniquement » déterminées. En effet on impose dans nos hypothèses que les S_i soient de variances unités, cela implique que les signaux réels (s'ils existent) seront égaux au signe près aux S_i . De même l'ordre des S_i est complètement aléatoire. Il faudra donc ajuster ces deux choses à la main à la fin de l'algorithme.

1. Dans cette question nous allons nous avancer un peu sur le cours de Ma321 (on justifiera et étendra le raisonnement qui va suivre dans ce module). Considérons une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ de classe $\mathcal{C}^1$. Fixons un $x \in \mathbb{R}^n$.
 - (a) Donner l'expression du développement de Taylor à l'ordre 1 de f en x dans la direction ϵh où $h \in \mathbb{R}^n$ et $\epsilon > 0$ est un réel « petit » i.e. donner l'expression à l'ordre 1 de :

$$f(x + \epsilon h) = ?$$

- (b) Justifier que lorsque $\epsilon > 0$ est suffisamment petit, en prenant $h = \nabla_x f$, le gradient de f en x (où l'on voit ce vecteur comme un vecteur colonne) on a :

$$f(x + \epsilon h) \geq f(x)$$

- (c) Du calcul précédent on en déduit un algorithme pour la recherche de maximum d'une fonction : l'algorithme du gradient à pas fixe.

Algorithme 1 Gradient à pas fixe (montée).

Fixons : $x^{(0)} \in \mathbb{R}^n$, $\epsilon > 0$ et $tol > 0$, $N \in \mathbb{N}^*$.

$x = x^{(0)}$

$nbiter = 0$

Tant que : $\|\nabla_x f\| > tol$ et $nbiter < N$:

$nbiter = nbiter + 1$

$x \leftarrow x + \epsilon \nabla_x f$

Retourner x

Programmer une fonction **grad_fix**(x_0, ϵ, tol, N) réalisant l'algorithme du gradient à pas fixe précédent. Attention cette fonction python dépend de la fonction f (par son gradient) considérée. Vous testerez votre algorithme sur les fonctions suivantes :

- i. Chercher un maximum de la fonction de $\mathbb{R}^2$ dans $\mathbb{R}$ définie par :

$$f(x) = -\frac{1}{2}(x_1^2 + 4x_2^2)$$

En utilisant la librairie Matplotlib et un tracé 3d vérifier empiriquement le résultat obtenu.

- ii. Chercher un minimum de la fonction de $\mathbb{R}^2$ dans $\mathbb{R}$ définie par :

$$f(x) = e^{x_1+3x_2+1} + e^{x_1-3x_2} + e^{-x_1+1}$$

Attention la méthode précédente sans une étude plus approfondie ne garantie pas la convergence, ni lorsque l'algorithme converge l'unicité du maximum (maximum local? maximum global?). Nous répondrons à ces questions dans le module Ma321.

2. Fixons $i \in \llbracket 1, m \rrbracket$. Nous allons utiliser l'algorithme précédent afin de maximiser le kurtosis. Dans la pratique nous n'avons pas accès aux lois de probabilités, seulement à des réalisations d'échantillons. Notons alors par abus Z la réalisation d'un n -échantillon de loi parente Z et par abus encore S la réalisation inconnue de S que nous cherchons à déterminer. On rappelle ainsi que les réalisations Z et S sont des matrices de tailles (m, n) . Nous devons donc maximiser les valeurs obtenues des estimateurs :

$$f(W_{i\bullet}) := \left| \widehat{kurt}(S_i) \right| = \left| \widehat{kurt}(W_{i\bullet}Z) \right|$$

- (a) Soit $j \in \llbracket 1, n \rrbracket$. Montrer que :

$$\frac{\partial f}{\partial W_{ij}}(W_{i\bullet}) = \text{sign}\left(\widehat{kurt}(S_i)\right) \times \frac{4}{n} \sum_{k=1}^n Z_{jk}(W_{i\bullet}Z_{\bullet k})^3$$

où $Z_{\bullet k}$ note la k -ième colonne de la réalisation Z , $W_{i\bullet}$ la i -ième ligne de la matrice W et Z_{jk} le coefficient Z_{jk} de la réalisation Z . En déduire l'expression du gradient :

$$\frac{\partial f}{\partial W_{i\bullet}}(W_{i\bullet}) := \left(\frac{\partial f}{\partial W_{i1}}(W_{i\bullet}), \dots, \frac{\partial f}{\partial W_{in}}(W_{i\bullet}) \right)$$

Comme $W_{i\bullet}$ est un vecteur ligne nous considérerons ce dernier vecteur comme un vecteur ligne.

- (b) Ainsi en partant d'une matrice orthogonale aléatoire $W = W^{(0)}$, on peut alors appliquer une itération de l'algorithme du gradient à pas fixe précédent pour chaque ligne i de W et obtenir une nouvelle matrice $\tilde{W}^{(1)}$. Or rien ne garantie que cette matrice soit également orthogonale. En utilisant un résultat obtenu dans le BE de Ma313, déterminer « la meilleure » approximation orthogonale de $\tilde{W}^{(1)}$ au sens de la norme de Frobenius i.e. résoudre le problème suivant :

$$\arg \min_{X^T X = I_n} \left\| \tilde{W}^{(1)} - X \right\|_F^2$$

3. **Synthèse :** Créer une fonction python résolvant le problème de l'ICA par maximisation du kurtosis. Vous appellerez votre fonction **ICA**($Z, W^{(0)}, \epsilon, \text{tol}, N$) où X est une matrice de taille (m, n) et $W^{(0)}$ une matrice orthogonale de taille (m, m) , ϵ et tol deux « petits » réels strictement positifs et $N \in \mathbb{N}^*$. Cette fonction réalisera l'algorithme suivant :

Algorithme 2 Analyse en composantes indépendantes (ICA) par maximisation du kurtosis.

Fixons : $W^{(0)} \in \mathcal{M}_{m,m}$ orthogonale, $\epsilon > 0$ et $\text{tol} > 0$, $N \in \mathbb{N}^*$.

Calculer $X^{(c)}$ la matrice centrée sur les lignes de X .

Calculer Z la matrice blanchie de $X^{(c)}$.

$nbiter = 0$

$W^{(-1)} = 0_{m,m}$

Tant que : $\left\| W^{(k)} - W^{(k-1)} \right\| > \text{tol}$ et $nbiter < N$:

$nbiter = nbiter + 1$

Pour tout $i \in \llbracket 1, m \rrbracket$:

Calculer $\frac{\partial f}{\partial W_{i\bullet}^{(k)}}(W_{i\bullet}^{(k)})$

$\tilde{W}_{i\bullet}^{(k+1)} \leftarrow W_{i\bullet}^{(k)} + \epsilon \frac{\partial f}{\partial W_{i\bullet}^{(k)}}(W_{i\bullet}^{(k)})$

$W^{(k+1)} = \arg \min_{X^T X = I_n} \left\| \tilde{W}^{(k)} - X \right\|_F^2$

$S = W^{(k_{final})} Z$

Retourner W, Z, S

Vous testerez votre algorithme sur l'exemple introductif avec $\epsilon = 0.01$ et $tol = 10^{-8}$. Pour cela charger grâce à la commande `np.load` le fichier de donnée **X_exemple1.npy**. Ce fichier contient une « hyper »-matrice de dimension (2, 250000, 3). Le paramètre de profondeur (le 3) indique les couches de couleurs. On pourra dans un premier temps se concentrer sur une unique couche (image en noir et blanc). Comme pour la partie, on pourra renormaliser les valeurs du S obtenu en utilisant la commande suivante avant la conversion en `uint8` :

```
Si=cv2.normalize(Si,None,np.min(x),np.max(x),cv2.NORM_MINMAX)
```

où S_i note une des images sources séparés (qui contient trois couches en couleurs, une seule en noir et blanc).

4. En réalisant vous même des mélanges linéaires de « signaux » comme des variables aléatoires simulées, des sons wav importés, des images vectorisées », etc... tester la capacité à séparer de votre algorithme.



FIGURE 8 – Exemple : en haut : mélange linéaire de deux images couleurs, en bas : résultat de l'ICA.

Pour les sons, on pourra tester la séparation sur le fichier « son_melange1.npy ». On pourra écouter le mélange sonore en utilisant les commandes suivantes :

```
import sounddevice as sd
fe=22050
X=np.load("son_melange1.npy")
Xrenorm=cv2.normalize(X,None,-15000,15000,cv2.NORM_MINMAX).astype("int16").T
sd.play(Xrenorm[:,0],fe) #pour X1 : sd.play(Xrenorm[:,1],fe) pour X2
```

Après convergence de votre algorithme on pourra écouter le résultat de la séparation grâce aux commandes (où S note la sortie S de l'algorithme) :

```
Srenorm=cv2.normalize(S,None,-15000,15000,cv2.NORM_MINMAX).astype("int16").T
sd.play(Srenorm[:,0],fe) #pour S1 : sd.play(Srenorm[:,1],fe) pour S2
```

5. **Ouverture** : dans la pratique il est courant d'avoir des vecteurs aléatoires X et S de différentes tailles. En particuliers le problème devient beaucoup plus complexe si l'on a un nombre de mélanges plus petit que le nombre de sources recherché. Essayer de traiter ce cas.